

E-commerce Marketing and Sales Analysis

A Data-Driven Exploration of Customer Acquisition, Retention, and Revenue Optimization

Author: Gangadhar Panda

Tool Used: Python (Jupyter Notebook)

ECOMMERCE MARKETING & SALES ANALYSIS NOTEBOOK

```
In [14]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
from matplotlib.ticker import FuncFormatter
from sklearn.preprocessing import StandardScaler
```

```
In [12]: #!gdown --folder https://drive.google.com/drive/folders/1Qt1HfSoTyCKiyDy2frR-hYOT9UvfwGq7?usp=sharing -O "/content/dr
```

```

Retrieving folder contents
Processing file 1rqihT647UW9HEmERxCrTJrcZZ9xE5TEi CustomersData.xlsx
Processing file 1u2gedRtVaCqTQNYlaohI9W06m3pA8rnc Dataset Description.docx
Processing file 1Cf0RUKckP7Qi9swmCe8XEgLLWnpKl4eE Discount_Coupon.csv
Processing file 1hjHgCluvPEUfrp9w-_-73ch0d9SK6FHF Marketing_Spend.csv
Processing file 1ZvQn7-UtGSdJa3H9zM4ve_uzACFAzF0Z Online_Sales.csv
Processing file 1tqJmtKcfhiEJXYVV21ybnD4UaXYW9r4v Tax_amount.xlsx
Retrieving folder contents completed
Building directory structure
Building directory structure completed
Downloading...
From: https://drive.google.com/uc?id=1rqihT647UW9HEmERxCrTJrcZZ9xE5TEi
To: /content/drive/My Drive/Colab_Downloads/CustomersData.xlsx
100% 42.2k/42.2k [00:00<00:00, 14.9MB/s]
Downloading...
From: https://drive.google.com/uc?id=1u2gedRtVaCqTQNYlaohI9W06m3pA8rnc
To: /content/drive/My Drive/Colab_Downloads/Dataset Description.docx
100% 9.06k/9.06k [00:00<00:00, 30.0MB/s]
Downloading...
From: https://drive.google.com/uc?id=1Cf0RUKckP7Qi9swmCe8XEgLLWnpKl4eE
To: /content/drive/My Drive/Colab_Downloads/Discount_Coupon.csv
100% 4.92k/4.92k [00:00<00:00, 21.8MB/s]
Downloading...
From: https://drive.google.com/uc?id=1hjHgCluvPEUfrp9w-_-73ch0d9SK6FHF
To: /content/drive/My Drive/Colab_Downloads/Marketing_Spend.csv
100% 8.67k/8.67k [00:00<00:00, 34.7MB/s]
Downloading...
From: https://drive.google.com/uc?id=1ZvQn7-UtGSdJa3H9zM4ve_uzACFAzF0Z
To: /content/drive/My Drive/Colab_Downloads/Online_Sales.csv
100% 5.24M/5.24M [00:00<00:00, 27.6MB/s]
Downloading...
From: https://drive.google.com/uc?id=1tqJmtKcfhiEJXYVV21ybnD4UaXYW9r4v
To: /content/drive/My Drive/Colab_Downloads/Tax_amount.xlsx
100% 9.84k/9.84k [00:00<00:00, 35.8MB/s]
Download completed

```

Read all datasets

```
In [50]: sales = pd.read_csv('/content/drive/My Drive/Colab_Downloads/Online_Sales.csv', parse_dates=['Transaction_Date'])
sales.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 52924 entries, 0 to 52923
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   CustomerID            52924 non-null  int64
1   Transaction_ID        52924 non-null  int64
2   Transaction_Date      52924 non-null  datetime64[ns]
3   Product_SKU           52924 non-null  object
4   Product_Description   52924 non-null  object
5   Product_Category     52924 non-null  object
6   Quantity              52924 non-null  int64
7   Avg_Price             52924 non-null  float64
8   Delivery_Charges     52924 non-null  float64
9   Coupon_Status        52924 non-null  object
dtypes: datetime64[ns](1), float64(2), int64(3), object(4)
memory usage: 4.0+ MB
```

```
In [51]: dfcustomers = pd.read_excel('/content/drive/My Drive/Colab_Downloads/CustomersData.xlsx')
dfcustomers.to_csv('/content/drive/My Drive/Colab_Downloads/CustomersData.csv', index=False)
customers = pd.read_csv('/content/drive/My Drive/Colab_Downloads/CustomersData.csv')
customers.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1468 entries, 0 to 1467
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   CustomerID      1468 non-null  int64
1   Gender          1468 non-null  object
2   Location        1468 non-null  object
3   Tenure_Months   1468 non-null  int64
dtypes: int64(2), object(2)
memory usage: 46.0+ KB
```

```
In [53]: coupons = pd.read_csv('/content/drive/My Drive/Colab_Downloads/Discount_Coupon.csv')
coupons.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 204 entries, 0 to 203
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Month           204 non-null   object
1   Product_Category 204 non-null   object
2   Coupon_Code      204 non-null   object
3   Discount_pct     204 non-null   int64
dtypes: int64(1), object(3)
memory usage: 6.5+ KB
```

```
In [54]: marketing = pd.read_csv('/content/drive/My Drive/Colab_Downloads/Marketing_Spend.csv', parse_dates=['Date'])
marketing.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 365 entries, 0 to 364
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Date            365 non-null   datetime64[ns]
1   Offline_Spend   365 non-null   int64
2   Online_Spend    365 non-null   float64
dtypes: datetime64[ns](1), float64(1), int64(1)
memory usage: 8.7 KB
```

```
In [55]: taxinfo = pd.read_excel('/content/drive/My Drive/Colab_Downloads/Tax_amount.xlsx')
taxinfo.to_csv('/content/drive/My Drive/Colab_Downloads/Tax_amount.csv', index=False)
tax = pd.read_csv('/content/drive/My Drive/Colab_Downloads/Tax_amount.csv')
tax.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Product_Category 20 non-null   object
1   GST              20 non-null   float64
dtypes: float64(1), object(1)
memory usage: 452.0+ bytes
```

```
In [56]: print("Sales rows:", len(sales))
print("Customers rows:", len(customers))
```



```
print("Marketing rows:", len(marketing))  
print("tax rows:", len(tax))  
print("coupons rows", len(coupons))
```

Sales rows: 52924
Customers rows: 1468
Marketing rows: 365
tax rows: 20
coupons rows 204

Clean and duplicate check ,normalize columns

```
In [57]: sales.columns = sales.columns.str.strip()  
customers.columns = customers.columns.str.strip()  
marketing.columns = marketing.columns.str.strip()  
coupons.columns = coupons.columns.str.strip()  
tax.columns = tax.columns.str.strip()
```

```
In [58]: # revenue per line (before tax): assume Avg_Price * Quantity - coupon discount + delivery charges  
sales['Revenue_Gross'] = sales['Avg_Price'] * sales['Quantity'] + sales['Delivery_Charges']  
# Mark coupon application flag  
sales['Coupon_Applied'] = sales['Coupon_Status'].notna() & (sales['Coupon_Status'] != '')  
# Transaction month and date parts  
sales['date'] = sales['Transaction_Date'].dt.date  
sales['month'] = sales['Transaction_Date'].dt.to_period('M').dt.to_timestamp()  
sales['month_str'] = sales['Transaction_Date'].dt.strftime('%Y-%m')  
  
# Ensure CustomerID is string  
sales['CustomerID'] = sales['CustomerID'].astype(str)  
customers['CustomerID'] = customers['CustomerID'].astype(str)  
  
sales.head()
```

Out[58]:

	CustomerID	Transaction_ID	Transaction_Date	Product_SKU	Product_Description	Product_Category	Quantity	Avg_Price
0	17850	16679	2019-01-01	GGOENEBJ079499	Nest Learning Thermostat 3rd Gen- USA - Stainle...	Nest-USA	1	153.71
1	17850	16680	2019-01-01	GGOENEBJ079499	Nest Learning Thermostat 3rd Gen- USA - Stainle...	Nest-USA	1	153.71
2	17850	16681	2019-01-01	GGOEGFKQ020399	Google Laptop and Cell Phone Stickers	Office	1	2.05
3	17850	16682	2019-01-01	GGOEGAAB010516	Google Men's 100% Cotton Short Sleeve Hero Tee...	Apparel	5	17.53
4	17850	16682	2019-01-01	GGOEGBJL013999	Google Canvas Tote Natural/Navy	Bags	1	16.50

```
In [59]: sales = sales.merge(customers[['CustomerID', 'Gender', 'Location', 'Tenure_Months']], on='CustomerID', how='left')
sales.head()
```

Out[59]:

	CustomerID	Transaction_ID	Transaction_Date	Product_SKU	Product_Description	Product_Category	Quantity	Avg_Price
0	17850	16679	2019-01-01	GGOENEBJ079499	Nest Learning Thermostat 3rd Gen- USA - Stainle...	Nest-USA	1	153.71
1	17850	16680	2019-01-01	GGOENEBJ079499	Nest Learning Thermostat 3rd Gen- USA - Stainle...	Nest-USA	1	153.71
2	17850	16681	2019-01-01	GGOEGFKQ020399	Google Laptop and Cell Phone Stickers	Office	1	2.05
3	17850	16682	2019-01-01	GGOEGAAB010516	Google Men's 100% Cotton Short Sleeve Hero Tee...	Apparel	5	17.53
4	17850	16682	2019-01-01	GGOEGBJL013999	Google Canvas Tote Natural/Navy	Bags	1	16.50

===== 3. ACQUISITION: monthly new customers =====

```
In [60]: # first purchase date per customer
first_purchase = sales.groupby('CustomerID')['Transaction_Date'].min().reset_index().rename(columns={'Transaction_Date': 'First_Purchase_Date'})
first_purchase['First_Purchase_Month'] = first_purchase['First_Purchase_Date'].dt.to_period('M').dt.to_timestamp()
acq_by_month = first_purchase.groupby('First_Purchase_Month').size().reset_index(name='New_Customers')
```

```
In [61]: # Plot acquisition
fig, ax = plt.subplots()
ax.plot(acq_by_month['First_Purchase_Month'], acq_by_month['New_Customers'], marker='o')
ax.set_title('Monthly New Customer Acquisitions')
ax.set_ylabel('New customers')
ax.set_xlabel('Month')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
# Highest and Lowest months
max_row = acq_by_month.loc[acq_by_month['New_Customers'].idxmax()]
min_row = acq_by_month.loc[acq_by_month['New_Customers'].idxmin()]
print("Highest acquisition month:", max_row['First_Purchase_Month'], "count:", int(max_row['New_Customers']))
print("Lowest acquisition month :", min_row['First_Purchase_Month'], "count:", int(min_row['New_Customers']))
```



Highest acquisition month: 2019-01-01 00:00:00 count: 215
 Lowest acquisition month : 2019-11-01 00:00:00 count: 68

===== 4. MONTHLY ACQUISITIONS PATTERN (seasonality) =====

```
In [62]: # Show month-over-month pattern
acq_by_month['month_name'] = acq_by_month['First_Purchase_Month'].dt.strftime('%b')
acq_by_month = acq_by_month.sort_values('First_Purchase_Month')
display(acq_by_month)
```

	First_Purchase_Month	New_Customers	month_name
0	2019-01-01	215	Jan
1	2019-02-01	96	Feb
2	2019-03-01	177	Mar
3	2019-04-01	163	Apr
4	2019-05-01	112	May
5	2019-06-01	137	Jun
6	2019-07-01	94	Jul
7	2019-08-01	135	Aug
8	2019-09-01	78	Sep
9	2019-10-01	87	Oct
10	2019-11-01	68	Nov
11	2019-12-01	106	Dec

===== 5. RETENTION / COHORT ANALYSIS =====

```
In [63]: # Create order month and cohort month
sales['order_month'] = sales['Transaction_Date'].dt.to_period('M').dt.to_timestamp()
# compute cohort (customer's first order month)
cohort = sales.groupby('CustomerID')['Transaction_Date'].min().reset_index().rename(columns={'Transaction_Date': 'CohortDate'})
cohort['CohortMonth'] = cohort['CohortDate'].dt.to_period('M').dt.to_timestamp()
sales = sales.merge(cohort[['CustomerID', 'CohortMonth']], on='CustomerID', how='left')
```

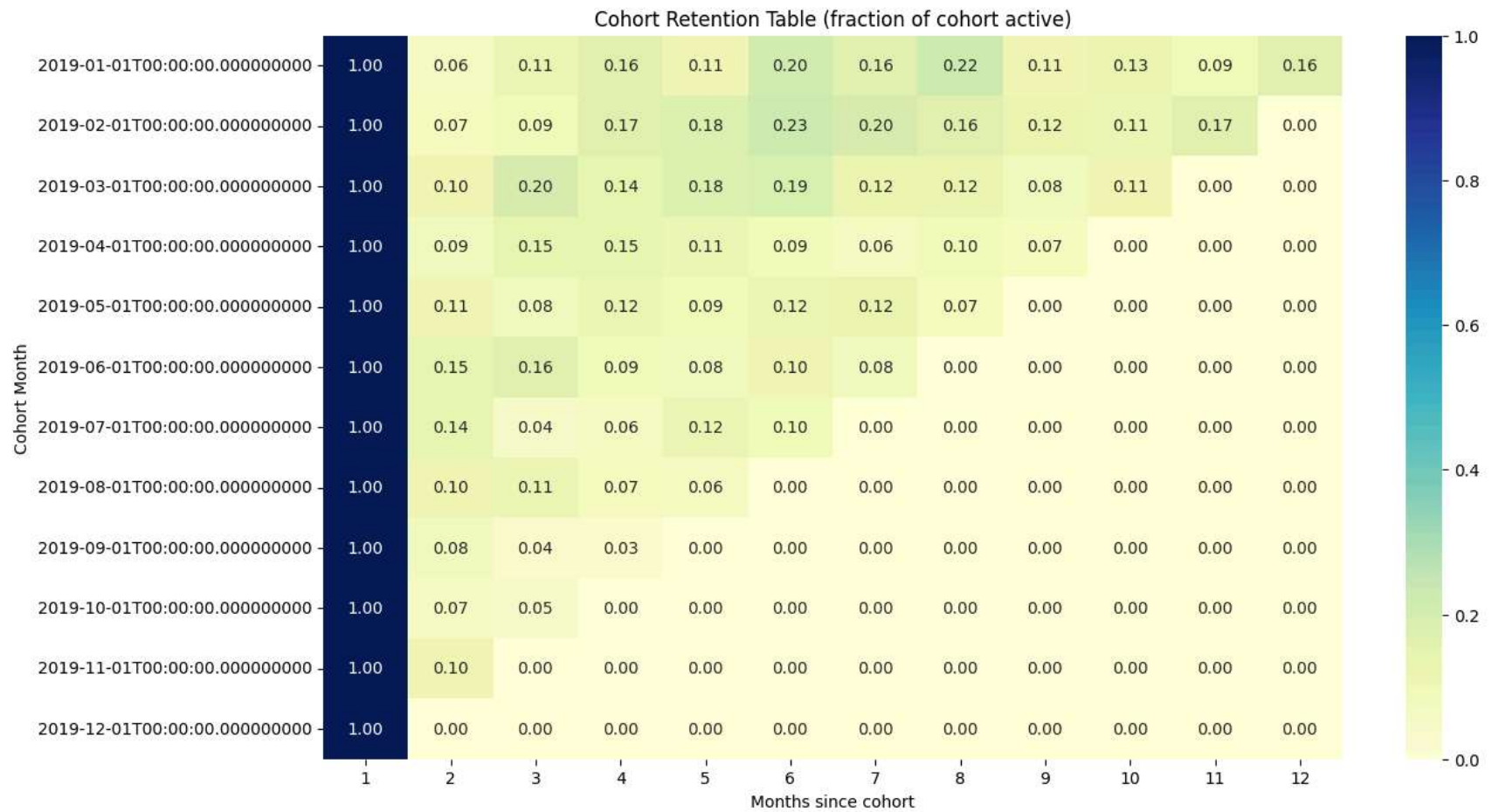
```
# Build cohort table: number of active customers per cohort per order_month
cohort_group = sales.groupby(['CohortMonth', 'order_month'])['CustomerID'].nunique().reset_index()
cohort_group['CohortIndex'] = ((cohort_group['order_month'].dt.year - cohort_group['CohortMonth'].dt.year) * 12 +
                              (cohort_group['order_month'].dt.month - cohort_group['CohortMonth'].dt.month) + 1)

cohort_pivot = cohort_group.pivot_table(index='CohortMonth', columns='CohortIndex', values='CustomerID').fillna(0)
cohort_sizes = cohort_pivot.iloc[:,0]
retention = cohort_pivot.divide(cohort_sizes, axis=0)
```

```
In [66]: # retention heatmap
plt.figure(figsize=(14,8))
sns.heatmap(retention, annot=True, fmt='.2f', cmap='YlGnBu')
plt.title('Cohort Retention Table (fraction of cohort active)')
plt.ylabel('Cohort Month')
plt.xlabel('Months since cohort')
plt.show()

# Identify strongest/weakest retention periods: compute average retention at month 2 or 3
retention_avg_by_cohort = retention.mean(axis=1)
best_cohort = retention_avg_by_cohort.idxmax()
worst_cohort = retention_avg_by_cohort.idxmin()
print("Best cohort (avg retention):", best_cohort)
print("Worst cohort (avg retention):", worst_cohort)

retention
```



Best cohort (avg retention): 2019-01-01 00:00:00

Worst cohort (avg retention): 2019-12-01 00:00:00

Out[66]:

CohortIndex	1	2	3	4	5	6	7	8	9	10	11	12
CohortMonth												
2019-01-01	1.0	0.060465	0.111628	0.158140	0.106977	0.204651	0.162791	0.218605	0.106977	0.130233	0.093023	0.15814
2019-02-01	1.0	0.072917	0.093750	0.166667	0.177083	0.229167	0.197917	0.156250	0.125000	0.114583	0.166667	0.00000
2019-03-01	1.0	0.101695	0.197740	0.141243	0.180791	0.186441	0.124294	0.124294	0.084746	0.107345	0.000000	0.00000
2019-04-01	1.0	0.085890	0.147239	0.147239	0.110429	0.092025	0.061350	0.098160	0.073620	0.000000	0.000000	0.00000
2019-05-01	1.0	0.107143	0.080357	0.116071	0.089286	0.116071	0.125000	0.071429	0.000000	0.000000	0.000000	0.00000
2019-06-01	1.0	0.145985	0.160584	0.087591	0.080292	0.102190	0.080292	0.000000	0.000000	0.000000	0.000000	0.00000
2019-07-01	1.0	0.138298	0.042553	0.063830	0.117021	0.095745	0.000000	0.000000	0.000000	0.000000	0.000000	0.00000
2019-08-01	1.0	0.103704	0.111111	0.074074	0.059259	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.00000
2019-09-01	1.0	0.076923	0.038462	0.025641	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.00000
2019-10-01	1.0	0.068966	0.045977	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.00000
2019-11-01	1.0	0.102941	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.00000
2019-12-01	1.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.00000

6. CUSTOMER BEHAVIOR DURING HIGH-RETENTION MONTHS

For months with top retention, analyze avg order value, coupon usage, category mix

Find top 2 cohorts by avg retention

```
In [86]: top_cohorts = retention_avg_by_cohort.sort_values(ascending=False).head(2).index
for c in top_cohorts:
    print("Cohort:", c)
    cohort_customers = cohort[cohort['CohortMonth']==c]['CustomerID']
```



```
df = sales[sales['CustomerID'].isin(cohort_customers) & (sales['order_month'] == c + pd.DateOffset(months=0))]
print(" Avg Revenue per order:", df['Revenue_Gross'].mean())
print(" Coupon usage %:", df['Coupon_Applied'].mean())

print(" Top categories:\n",df.groupby('Product_Category')['Revenue_Gross'].sum().sort_values(ascending=False).head())
```

Cohort: 2019-01-01 00:00:00

Avg Revenue per order: 113.922446468127

Coupon usage %: 1.0

Cohort: 2019-02-01 00:00:00

Avg Revenue per order: 110.56001756234633

Coupon usage %: 1.0

Top categories:

Product_Category

Nest-USA 174417.76

Apparel 44611.59

Office 26919.09

Bags 20872.67

Drinkware 18020.37

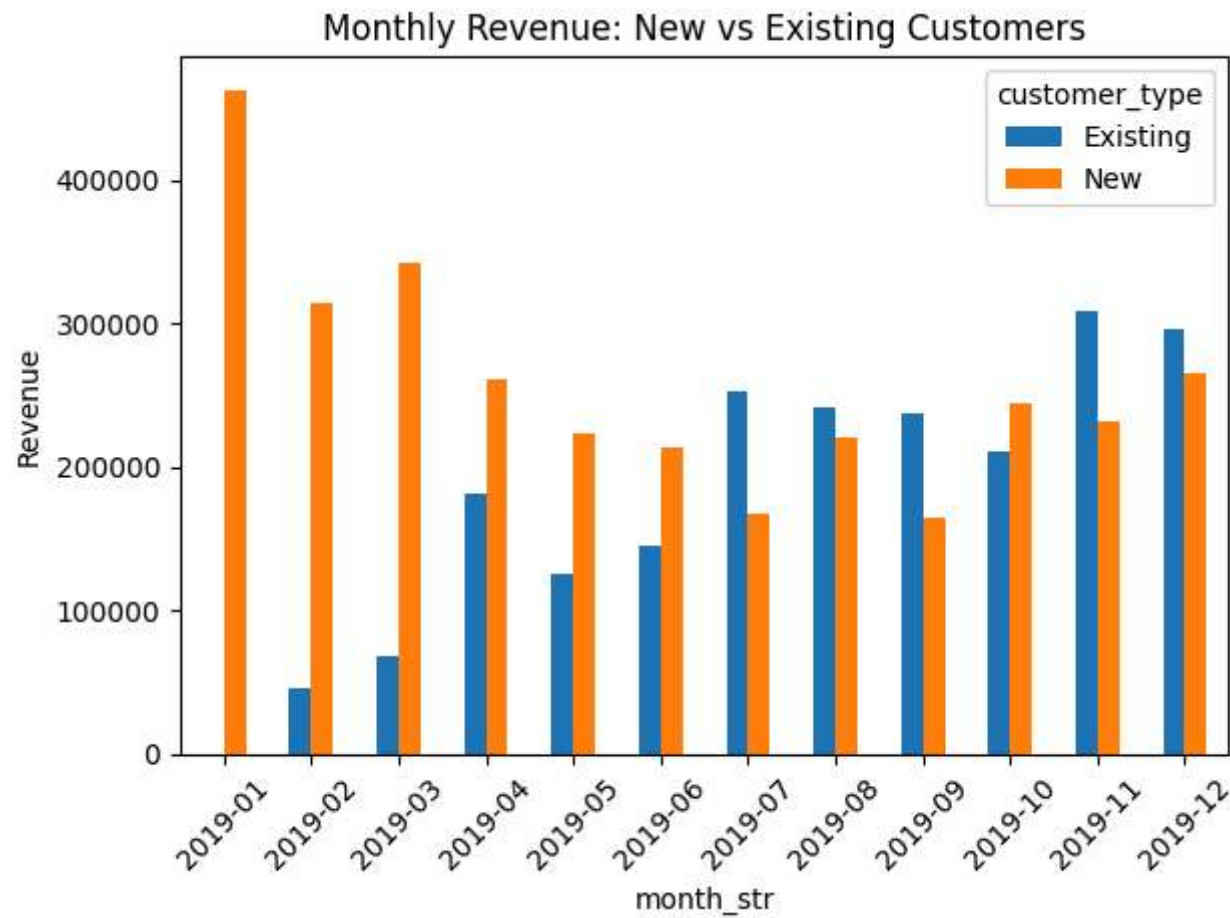
Name: Revenue_Gross, dtype: float64

7. NEW VS EXISTING CUSTOMER MONTHLY REVENUE

=====

```
In [87]: # Mark new/existing per month
# For each transaction, check if transaction date's month equals customer's first purchase month => new else existing
sales = sales.merge(first_purchase[['CustomerID', 'First_Purchase_Date']], on='CustomerID', how='left')
sales['first_purchase_month'] = sales['First_Purchase_Date'].dt.to_period('M').dt.to_timestamp()
sales['customer_type'] = np.where(sales['order_month'] == sales['first_purchase_month'], 'New', 'Existing')
```

```
In [88]: rev_monthly = sales.groupby(['month_str', 'customer_type'])['Revenue_Gross'].sum().reset_index()
rev_pivot = rev_monthly.pivot(index='month_str', columns='customer_type', values='Revenue_Gross').fillna(0)
rev_pivot.plot(kind='bar', stacked=False)
plt.title('Monthly Revenue: New vs Existing Customers')
plt.ylabel('Revenue')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



===== 8. COUPON USAGE & REVENUE =====

```
In [93]: coupon_summary = sales.groupby('month_str').agg({
    'Revenue_Gross': 'sum',
    'Coupon_Applied': 'sum'
}).rename(columns={'Coupon_Applied': 'Coupon_Count'})
coupon_summary['Coupon_Rate'] = coupon_summary['Coupon_Count'] / sales.groupby('month_str')['Transaction_ID'].nunique()
coupon_summary = coupon_summary.fillna(0)
display(coupon_summary)
```

	Revenue_Gross	Coupon_Count	Coupon_Rate
month_str			
2019-01	462866.90	4063	1.932921
2019-02	360036.40	3284	1.973558
2019-03	410408.03	4346	2.182823
2019-04	443100.16	4150	2.289024
2019-05	349159.59	4572	2.247788
2019-06	358594.96	4193	2.161340
2019-07	421362.00	5251	2.524519
2019-08	462309.94	6150	2.547639
2019-09	401553.82	4288	2.219462
2019-10	455643.16	4164	1.959529
2019-11	541254.55	3961	1.735758
2019-12	561140.18	4502	1.677347

===== 9. TOP-PERFORMING PRODUCTS

```
In [98]: sales['Product_SKU'] = sales['Product_SKU'].astype(str)

sales.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 52924 entries, 0 to 52923
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype
---  -
0   CustomerID            52924 non-null  object
1   Transaction_ID        52924 non-null  int64
2   Transaction_Date      52924 non-null  datetime64[ns]
3   Product_SKU           52924 non-null  object
4   Product_Description   52924 non-null  object
5   Product_Category     52924 non-null  object
6   Quantity              52924 non-null  int64
7   Avg_Price             52924 non-null  float64
8   Delivery_Charges     52924 non-null  float64
9   Coupon_Status        52924 non-null  object
10  Revenue_Gross         52924 non-null  float64
11  Coupon_Applied       52924 non-null  bool
12  date                  52924 non-null  object
13  month                 52924 non-null  datetime64[ns]
14  month_str             52924 non-null  object
15  Gender                52924 non-null  object
16  Location              52924 non-null  object
17  Tenure_Months        52924 non-null  int64
18  order_month           52924 non-null  datetime64[ns]
19  CohortMonth           52924 non-null  datetime64[ns]
20  First_Purchase_Date  52924 non-null  datetime64[ns]
21  first_purchase_month  52924 non-null  datetime64[ns]
22  customer_type         52924 non-null  object
dtypes: bool(1), datetime64[ns](6), float64(3), int64(3), object(10)
memory usage: 8.9+ MB
```

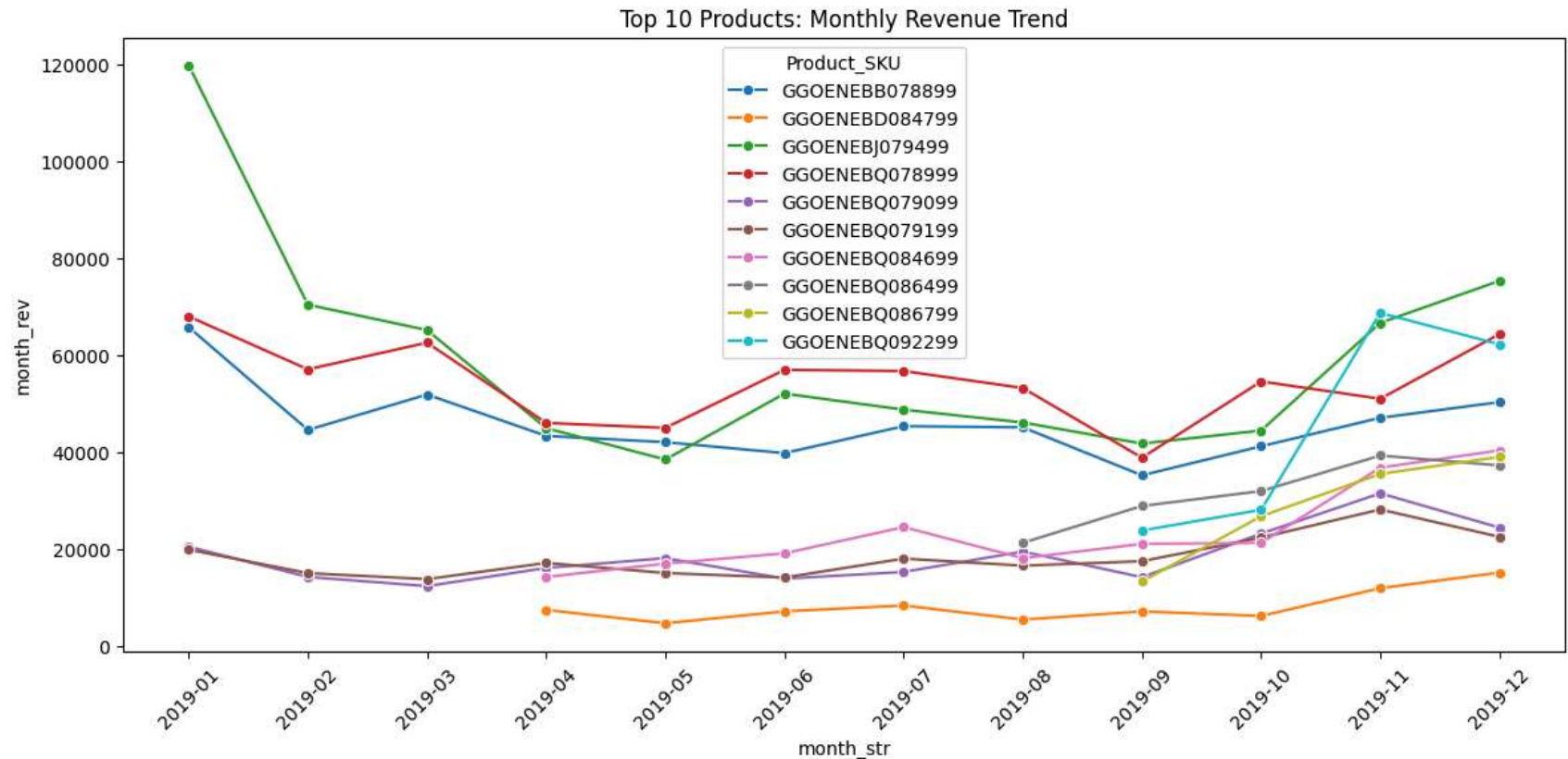
```
In [103... from os import PRIIO_PGRP
prod = sales.groupby(['Product_SKU', 'Product_Description', 'Product_Category']).agg(
    total_rev=('Revenue_Gross', 'sum'),
    total_qty=('Quantity', 'sum'),
    orders=('Transaction_ID', 'nunique')
).reset_index().sort_values('total_rev', ascending=False)

display(prod.head(20))
```

	Product_SKU	Product_Description	Product_Category	total_rev	total_qty	orders
981	GGOENEBJ079499	Nest Learning Thermostat 3rd Gen-USA - Stainle...	Nest-USA	713873.03	4570	3511
983	GGOENEBQ078999	Nest Cam Outdoor Security Camera - USA	Nest-USA	654298.21	5206	3328
976	GGOENEBB078899	Nest Cam Indoor Security Camera - USA	Nest-USA	551596.34	4402	3230
984	GGOENEBQ079099	Nest Protect Smoke + CO White Battery Alarm-USA	Nest-USA	223078.94	2683	1361
985	GGOENEBQ079199	Nest Protect Smoke + CO White Wired Alarm-USA	Nest-USA	219973.10	2670	1065
989	GGOENEBQ084699	Nest Learning Thermostat 3rd Gen-USA - White	Nest-USA	212370.22	1368	1089
994	GGOENEBQ092299	Nest Secure Alarm System Starter Pack - USA	Nest	182867.18	510	498
990	GGOENEBQ086499	Nest Cam IQ - USA	Nest	158634.21	771	599
992	GGOENEBQ086799	Nest Thermostat E - USA	Nest	114530.70	1091	844
980	GGOENEBD084799	Nest Learning Thermostat 3rd Gen-USA - Copper	Nest-USA	73308.22	472	393
995	GGOENEBQ092399	Nest Cam IQ Outdoor - USA (Preorder)	Nest	63856.78	225	148
969	GGOEGOLC014299	Google Metallic Notebook Set	Office	40801.26	6496	378
892	GGOEGDHQ015399	26 oz Double Wall Insulated Bottle	Drinkware	39712.59	1567	408
952	GGOEGOCB017499	Leatherette Journal	Office	38143.52	3741	220
854	GGOEGBMJ013399	Sport Bag	Bags	37964.26	7321	408
882	GGOEGDHC074099	Google 17oz Stainless Steel Sport Bottle	Drinkware	35764.14	1857	250
880	GGOEGDHC018299	Google 22 oz Water Bottle	Drinkware	34470.98	9728	583
961	GGOEGOCL077699	Google Hard Cover Journal	Notebooks & Journals	33802.19	2209	130
982	GGOENEBJ081899	Nest Learning Thermostat 3rd Gen - CA - Stainl...	Nest-Canada	28273.54	136	109
926	GGOEGHPJ080310	Google Blackout Cap	Headgear	27495.11	1903	264

```
In [104... # For top products show their monthly sales trend
top10 = prod.head(10)['Product_SKU'].tolist()
top_sales = sales[sales['Product_SKU'].isin(top10)].groupby(['Product_SKU', 'month_str']).agg(month_rev=('Revenue_Gros
```

```
plt.figure(figsize=(14,6))
sns.lineplot(data=top_sales, x='month_str', y='month_rev', hue='Product_SKU', marker='o')
plt.xticks(rotation=45)
plt.title('Top 10 Products: Monthly Revenue Trend')
plt.show()
```



===== 10. MARKETING SPEND VS REVENUE (MONTHLY) =====

```
In [139... marketing['month'] = marketing['Date'].dt.to_period('M').dt.to_timestamp()
monthly_spend = marketing.groupby('month').agg(offline_spend=('Offline_Spend','sum'), online_spend=('Online_Spend','sum'))
monthly_rev = sales.groupby('order_month').agg(monthly_revenue=('Revenue_Gross','sum')).reset_index().rename(columns={'monthly_revenue': 'month_rev'})

mkt_rev = monthly_spend.merge(monthly_rev, on='month', how='left').fillna(0)
```

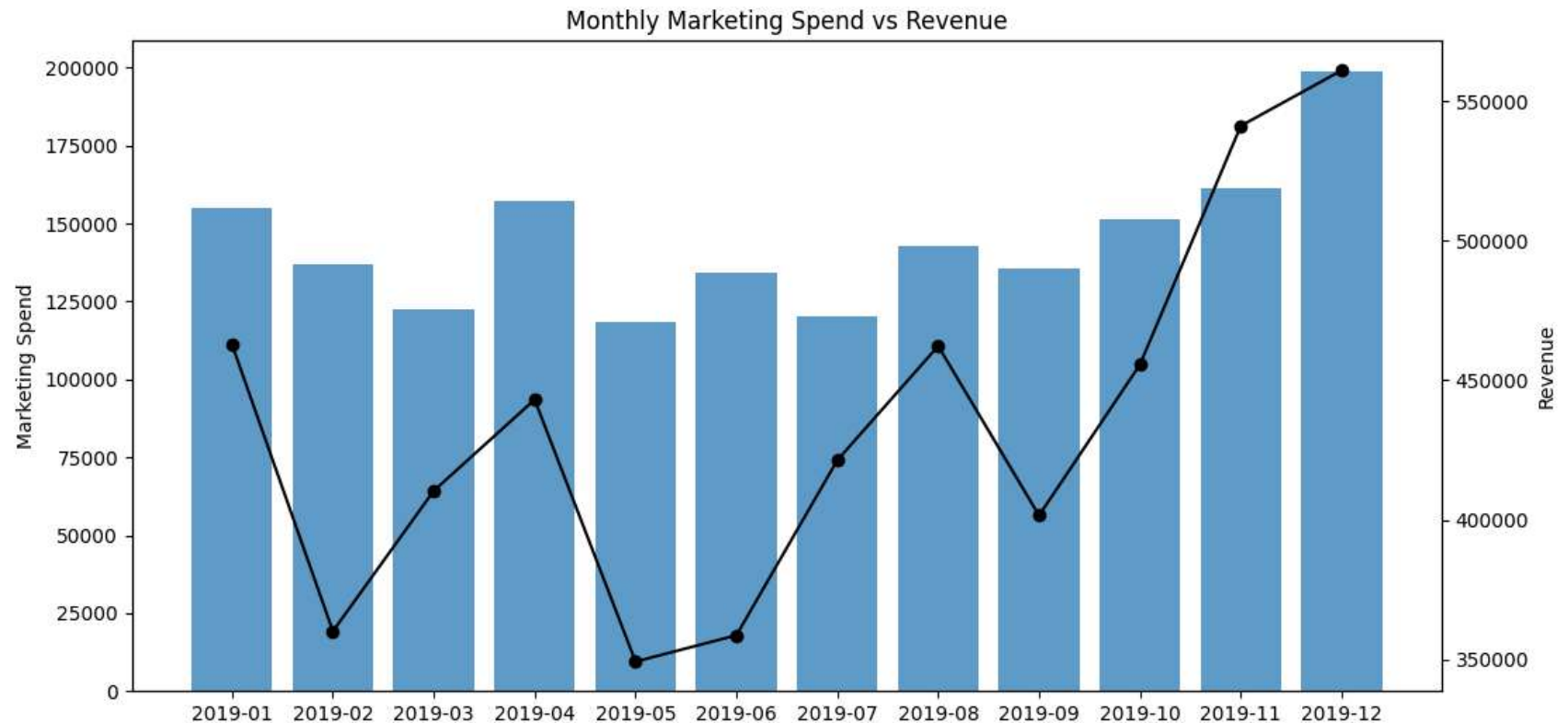
```

mkt_rev['total_spend'] = mkt_rev['offline_spend'] + mkt_rev['online_spend']
mkt_rev['ROI'] = np.where(mkt_rev['total_spend']>0, mkt_rev['monthly_revenue'] / mkt_rev['total_spend'], np.nan)

# Plot spend vs revenue
fig, ax1 = plt.subplots(figsize=(12, 6))
ax1.bar(mkt_rev['month'].dt.strftime('%Y-%m'), mkt_rev['total_spend'], label='Total Spend', alpha=0.7)
ax1.set_ylabel('Marketing Spend')
ax2 = ax1.twinx()
ax2.plot(mkt_rev['month'].dt.strftime('%Y-%m'), mkt_rev['monthly_revenue'], color='black', marker='o', label='Revenue')
ax2.set_ylabel('Revenue')
plt.xticks(rotation=180)
plt.title('Monthly Marketing Spend vs Revenue')
plt.show()

display(mkt_rev.sort_values('ROI', ascending=False))

```



	month	offline_spend	online_spend	monthly_revenue	total_spend	ROI
6	2019-07-01	67500	52717.85	421362.00	120217.85	3.504987
10	2019-11-01	93000	68144.96	541254.55	161144.96	3.358805
2	2019-03-01	73500	48750.09	410408.03	122250.09	3.357118
7	2019-08-01	85500	57404.15	462309.94	142904.15	3.235105
9	2019-10-01	93500	57724.65	455643.16	151224.65	3.013022
0	2019-01-01	96600	58328.95	462866.90	154928.95	2.987608
8	2019-09-01	83000	52514.54	401553.82	135514.54	2.963179
4	2019-05-01	65500	52759.64	349159.59	118259.64	2.952483
11	2019-12-01	122000	76648.75	561140.18	198648.75	2.824786
3	2019-04-01	96000	61026.83	443100.16	157026.83	2.821812
5	2019-06-01	80500	53818.14	358594.96	134318.14	2.669743
1	2019-02-01	81300	55807.92	360036.40	137107.92	2.625934

CAC (Customer Acquisition Cost) & ROAS (Return on Ad Spend)

```
In [211... # CAC = marketing_spend / new_customers
marketing['Total_Spend'] = marketing['Offline_Spend'] + marketing['Online_Spend']
marketing['month_str'] = marketing['month'].dt.strftime('%b-%Y')
acq_by_month['month_str'] = acq_by_month['First_Purchase_Month'].dt.strftime('%b-%Y')

mkt = (
    marketing.groupby('month_str')
    .agg(marketing_spend=('Total_Spend', 'sum'))
    .reset_index()
)

mkt = mkt.merge(acq_by_month[['month_str', 'New_Customers']], on='month_str', how='left')
mkt['CAC'] = mkt['marketing_spend'] / mkt['New_Customers']
```



```
print("Average CAC:", mkt['CAC'].mean())

rev_monthly['month'] = pd.to_datetime(rev_monthly['month_str'], errors='coerce')
rev_monthly['month_str'] = rev_monthly['month'].dt.strftime('%b-%Y')

mkt = mkt.merge(rev_monthly[['month_str', 'Revenue_Gross']], on='month_str', how='left')
mkt['ROAS'] = mkt['Revenue_Gross'] / mkt['marketing_spend']
display(mkt.sort_values('ROAS', ascending=False))
```

Average CAC: 1324.6684172455957

```
/tmp/ipython-input-2188401398.py:17: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  rev_monthly['month'] = pd.to_datetime(rev_monthly['month_str'], errors='coerce')
```

	month_str	marketing_spend	New_Customers	CAC	Revenue_Gross	ROAS
8	Jan-2019	154928.95	215	720.599767	462866.90	2.987608
14	Mar-2019	122250.09	177	690.678475	342113.64	2.798474
7	Feb-2019	137107.92	96	1428.207500	314764.37	2.295742
9	Jul-2019	120217.85	94	1278.913298	253328.88	2.107248
17	Nov-2019	161144.96	68	2369.778824	309487.80	1.920555
16	May-2019	118259.64	112	1055.889643	223157.62	1.887014
21	Sep-2019	135514.54	78	1737.365897	236956.08	1.748566
2	Aug-2019	142904.15	135	1058.549259	241454.08	1.689623
1	Apr-2019	157026.83	163	963.354785	261939.49	1.668119
20	Oct-2019	151224.65	87	1738.214368	244227.80	1.615000
12	Jun-2019	134318.14	137	980.424380	213946.91	1.592837
3	Aug-2019	142904.15	135	1058.549259	220855.86	1.545482
4	Dec-2019	198648.75	106	1874.044811	295812.76	1.489125
18	Nov-2019	161144.96	68	2369.778824	231766.75	1.438250
19	Oct-2019	151224.65	87	1738.214368	211415.36	1.398022
10	Jul-2019	120217.85	94	1278.913298	168033.12	1.397739
5	Dec-2019	198648.75	106	1874.044811	265327.42	1.335661
22	Sep-2019	135514.54	78	1737.365897	164597.74	1.214613
0	Apr-2019	157026.83	163	963.354785	181160.67	1.153692
11	Jun-2019	134318.14	137	980.424380	144648.05	1.076906
15	May-2019	118259.64	112	1055.889643	126001.97	1.065469
13	Mar-2019	122250.09	177	690.678475	68294.39	0.558645

	month_str	marketing_spend	New_Customers	CAC	Revenue_Gross	ROAS
6	Feb-2019	137107.92	96	1428.207500	45272.03	0.330193

===== 11. RFM SEGMENTATION =====

```
In [113... # Recency, Frequency, Monetary as of analysis date (end of dataset)
snapshot_date = sales['Transaction_Date'].max() + pd.DateOffset(days=1)
rfm = sales.groupby('CustomerID').agg(
    Recency = ('Transaction_Date', lambda x: (snapshot_date - x.max()).days),
    Frequency = ('Transaction_ID', 'nunique'),
    Monetary = ('Revenue_Gross', 'sum')
).reset_index()

# Score RFM into quartiles (1-4)
rfm['R_Score'] = pd.qcut(rfm['Recency'], 4, labels=[4,3,2,1]).astype(int) # more recent -> higher score
rfm['F_Score'] = pd.qcut(rfm['Frequency'].rank(method='first'), 4, labels=[1,2,3,4]).astype(int)
rfm['M_Score'] = pd.qcut(rfm['Monetary'], 4, labels=[1,2,3,4]).astype(int)
rfm['RFM_Score'] = rfm['R_Score'].map(str) + rfm['F_Score'].map(str) + rfm['M_Score'].map(str)
```

```
In [114... # Map to segments (simple)
def rfm_segment(row):
    if row['R_Score']>=3 and row['F_Score']>=3 and row['M_Score']>=3:
        return 'Premium'
    if row['F_Score']>=3:
        return 'Gold'
    if row['M_Score']>=3:
        return 'Silver'
    return 'Standard'

rfm['Segment'] = rfm.apply(rfm_segment, axis=1)
segment_summary = rfm.groupby('Segment').agg(
    customers=('CustomerID', 'nunique'),
    total_revenue=('Monetary', 'sum'),
    avg_revenue=('Monetary', 'mean')
).sort_values('total_revenue', ascending=False).reset_index()
display(segment_summary)
```

	Segment	customers	total_revenue	avg_revenue
0	Premium	426	3191094.68	7490.832582
1	Gold	308	1362237.12	4422.847792
2	Standard	671	497264.41	741.079598
3	Silver	63	176833.48	2806.880635

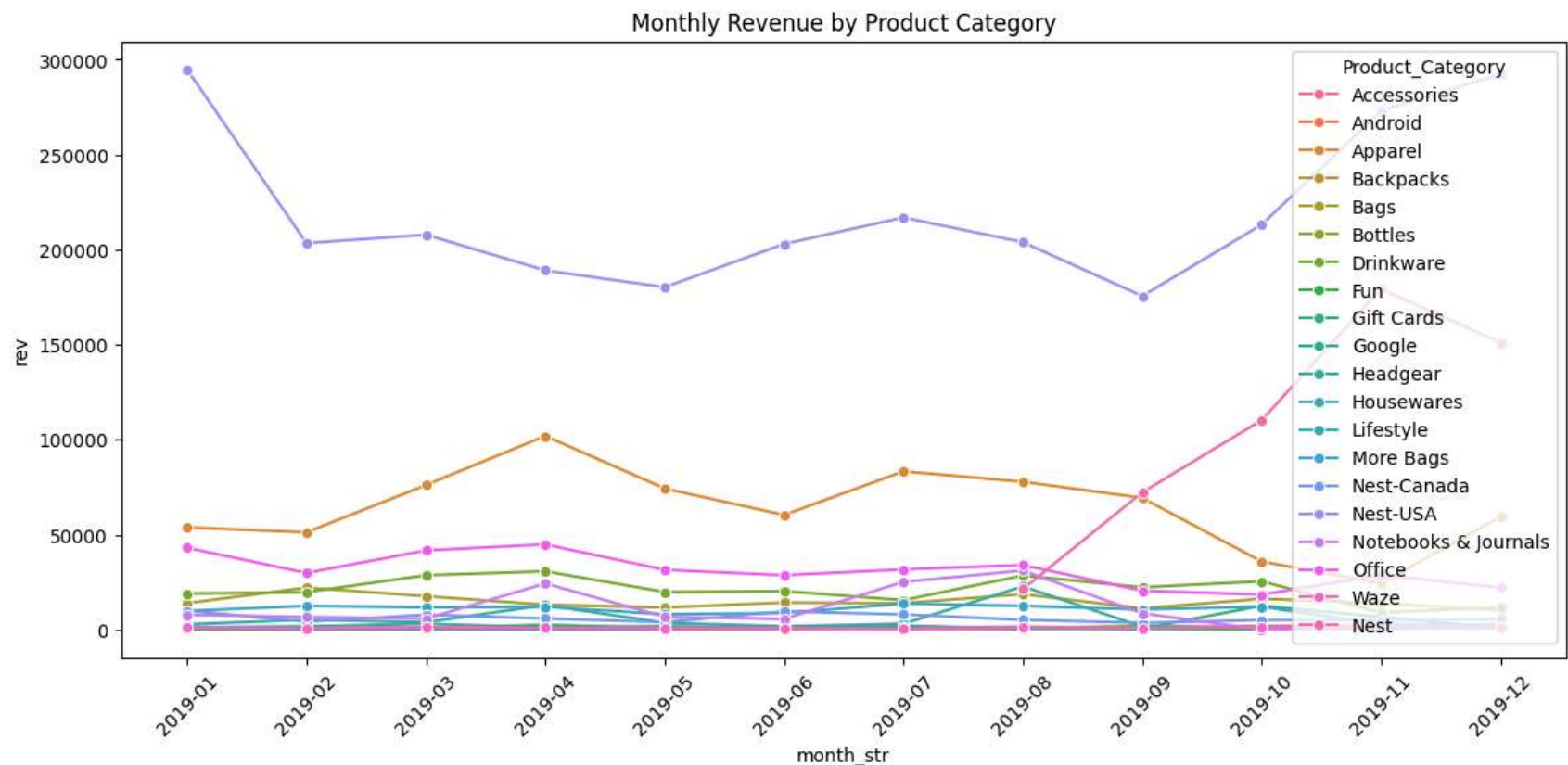
===== 12. COHORT LTV (cumulative revenue per cohort per month) =====

```
In [115... cohort_revenue = sales.groupby(['CohortMonth', 'order_month']).agg(rev=('Revenue_Gross', 'sum'), customers=('CustomerID', 'count'))
cohort_revenue['period_index'] = ((cohort_revenue['order_month'].dt.year - cohort_revenue['CohortMonth'].dt.year)*12 +
                                   (cohort_revenue['order_month'].dt.month - cohort_revenue['CohortMonth'].dt.month))
cohort_rev_pivot = cohort_revenue.pivot_table(index='CohortMonth', columns='period_index', values='rev').fillna(0).cumsum()
cohort_size = sales.groupby('CohortMonth')['CustomerID'].nunique()
cohort_ltv = cohort_rev_pivot.divide(cohort_size, axis=0)
display(cohort_ltv.head())
```

period_index	1	2	3	4	5	6	7	8	
CohortMonth									
2019-01-01	2152.869302	2363.436884	2635.727628	3204.199070	3364.555163	3637.573767	4070.609535	4339.916233	4552.5109
2019-02-01	3278.795521	3380.377604	3529.660729	3791.256354	4067.836042	4562.074583	4849.004375	5365.875104	5558.2795
2019-03-01	1932.845424	2184.868757	2414.908531	2604.631243	2903.201977	3345.131073	3642.325593	3888.056893	4037.8180
2019-04-01	1606.990736	1764.629939	1877.052209	2082.395215	2260.219325	2461.906687	2567.696564	2936.281902	3057.2526
2019-05-01	1992.478750	2059.368482	2184.417589	2337.784911	2505.538304	2867.381696	3226.565536	3336.446161	3336.4461

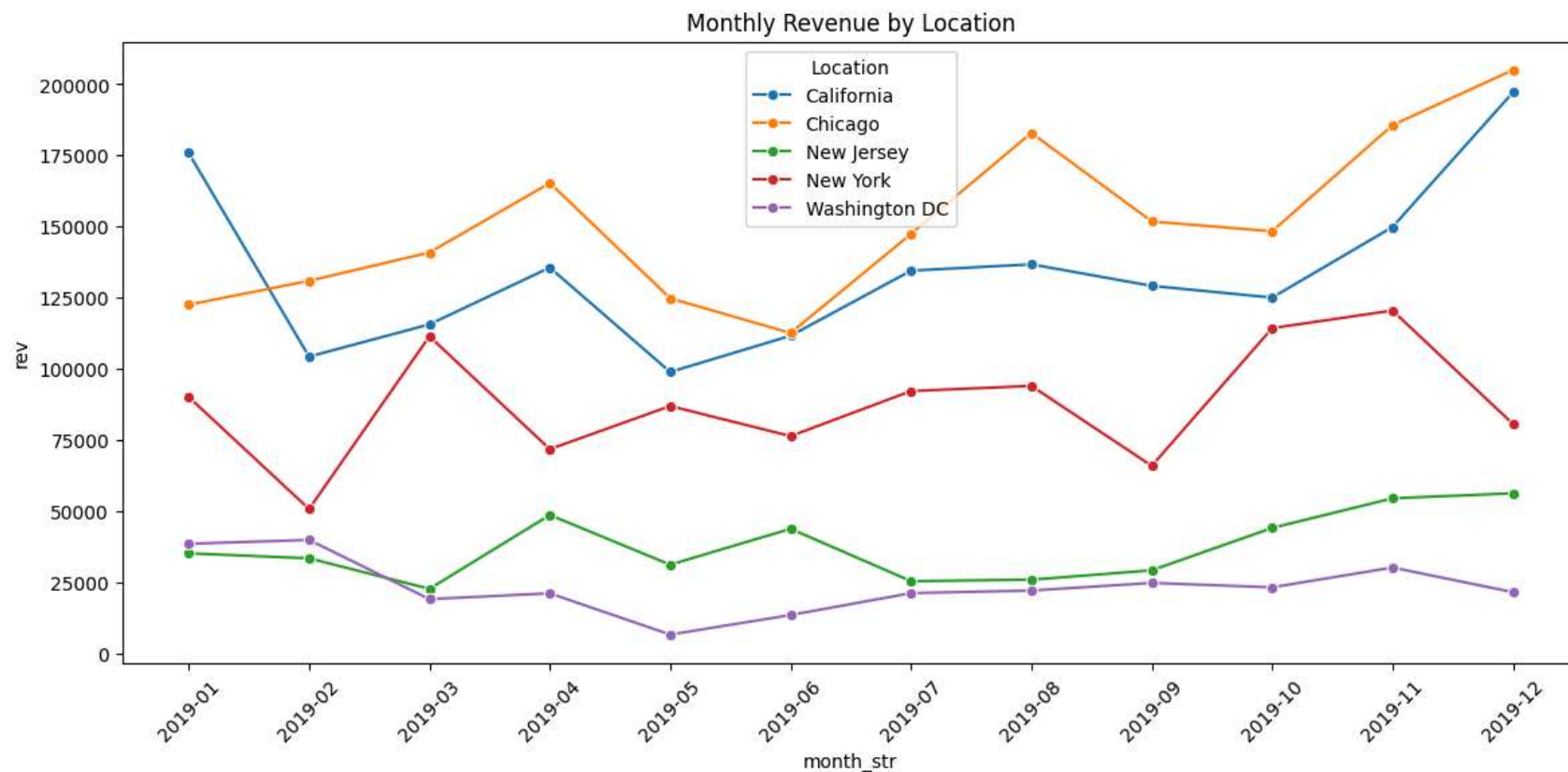
===== 13. SEASONAL TRENDS BY CATEGORY & LOCATION =====from os

```
In [122... from os import PRIO_PGRP
cat_month = sales.groupby(['month_str', 'Product_Category']).agg(rev=('Revenue_Gross', 'sum')).reset_index()
plt.figure(figsize=(14,6))
sns.lineplot(data=cat_month, x='month_str', y='rev', hue='Product_Category', marker='o')
plt.title('Monthly Revenue by Product Category')
plt.xticks(rotation=45)
plt.show()
```



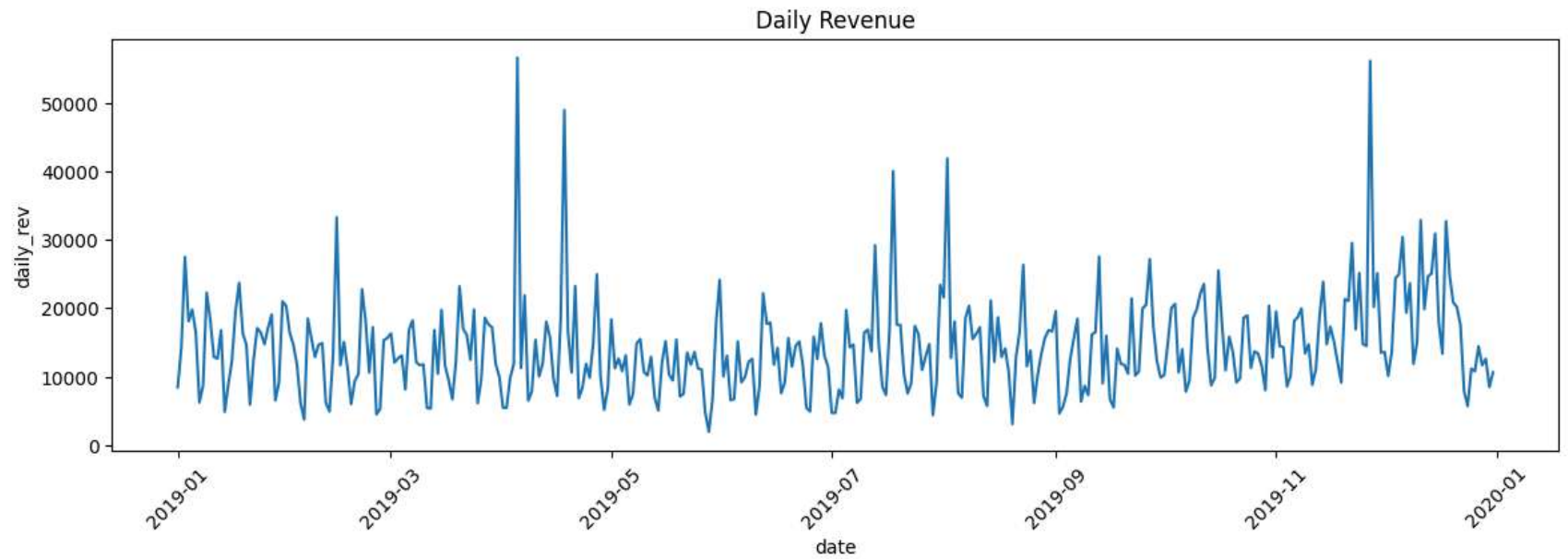
```
In [123... loc_month = sales.groupby(['month_str', 'Location']).agg(rev=('Revenue_Gross', 'sum')).reset_index()
plt.figure(figsize=(14,6))
```

```
sns.lineplot(data=loc_month, x='month_str', y='rev', hue='Location', marker='o')
plt.title('Monthly Revenue by Location')
plt.xticks(rotation=45)
plt.show()
```

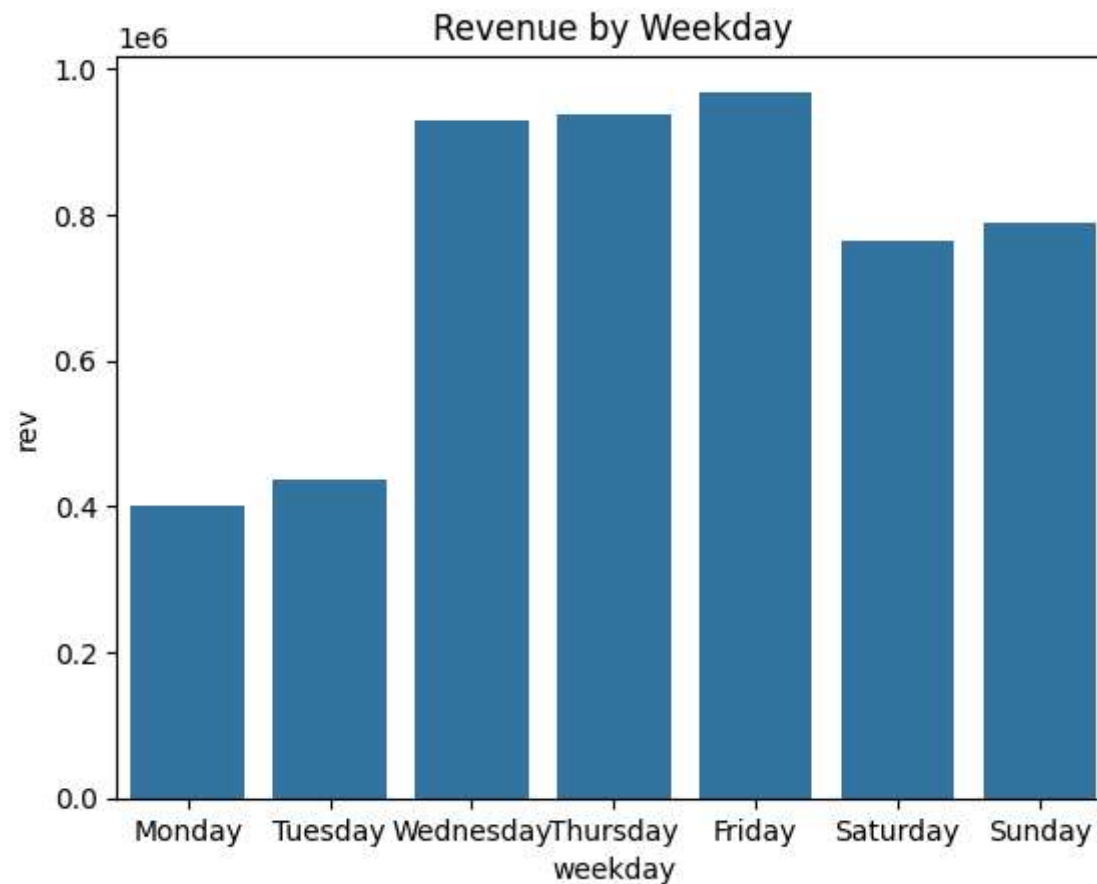


===== 14. DAILY & WEEKDAY TRENDS =====

```
In [124... sales['weekday'] = sales['Transaction_Date'].dt.day_name()
daily = sales.groupby('date').agg(daily_rev=('Revenue_Gross', 'sum')).reset_index()
plt.figure(figsize=(14,4))
sns.lineplot(data=daily, x='date', y='daily_rev')
plt.title('Daily Revenue')
plt.xticks(rotation=45)
plt.show()
```



```
In [128... weekday = sales.groupby('weekday').agg(rev=('Revenue_Gross', 'sum'), orders=('Transaction_ID', 'nunique')).reset_index()
# order weekdays
weekday['weekday_order'] = pd.Categorical(weekday['weekday'], categories=['Monday', 'Tuesday', 'Wednesday', 'Thursday',
weekday = weekday.sort_values('weekday_order')
sns.barplot(data=weekday, x='weekday', y='rev')
plt.title('Revenue by Weekday')
plt.show()
```



===== 15. OUTPUTS: Save key reports

```
In [133... import os
folder_path = '/content/drive/My Drive/Colab_Downloads/OUTPUT'
os.makedirs(folder_path, exist_ok=True)

acq_by_month.to_csv('/content/drive/My Drive/Colab_Downloads/OUTPUT/acquisition_by_month.csv', index=False)
retention.to_csv('/content/drive/My Drive/Colab_Downloads/OUTPUT/cohort_retention_matrix.csv')
rfm.to_csv('/content/drive/My Drive/Colab_Downloads/OUTPUT/rfm_segments.csv', index=False)
```



```
prod.head(100).to_csv('/content/drive/My Drive/Colab_Downloads/OUTPUT/top_products.csv', index=False)  
mkt_rev.to_csv('/content/drive/My Drive/Colab_Downloads/OUTPUT/marketing_vs_revenue.csv', index=False)
```

In []: